Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 270 (2025) 2205–2214

Procedia Computer Science

www.elsevier.com/locate/procedia

29th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2025)

Semantic Feedback Processing with LLMs: Automating Issue Detection and Prioritization in DevOps

Ciprian Paduraru[a], Miruna Zavelca[b], Alin Stefanescu[a,b]

[a] *University of Bucharest, Romania*
[b] *Institute for Logic and Data Science, Romania*

Abstract

This paper introduces an LLM-augmented pipeline for automating the transformation of unstructured Reddit feedback into structured Jira issues. Designed for DevOps environments, the system performs a two-layered sentiment analysis - combining fast, lightweight classification with high-accuracy LLM-based scoring - followed by LLM-driven summarization and component-aware semantic matching. Feedback is either linked to existing issues or triggers new ticket creation based on similarity thresholds and component alignment. A hybrid graph-relational backend supports issue clustering, trend detection, and historical traceability. Based on real-world game development data, the system shows high scalability and significantly improves triage efficiency and developer feedback coverage.

© 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the KES International.

Keywords: Large Language Models (LLMs); User Feedback Mining; Issue Tracking Automation; Semantic Similarity; Component Classification; DevOps Integration; Natural Language Processing (NLP)

1. Introduction

Large-scale software systems, particularly in consumer-facing domains like gaming, rely increasingly on real-time feedback to guide development priorities, fix critical bugs, and respond to user sentiment. However, the transformation of unstructured user input, such as social media discussions, into structured, developer-actionable tasks is labor-intensive and error-prone. As DevOps pipelines emphasize automation, traceability, and rapid iteration, there is growing demand for AI-powered systems that can ingest, analyze, and operationalize user feedback at scale.

This paper presents a multi-stage, DevOps-integrated pipeline that automates the detection, classification, and creation of software issues from unstructured Reddit feedback. The system uses a hybrid architecture combining fast keyword and sentiment-based filtering with transformer-based semantic reasoning. It leverages a two-stage sentiment analysis approach and uses a Large Language Model (LLM) for summarization and component-aware classification. Feedback is either matched with existing Jira issues using semantic similarity or used to generate new tickets priori-

1877-0509 © 2025 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the KES International.
10.1016/j.procs.2025.09.341

tized by urgency and recurrence. To support deduplication, feedback-to-issue mappings are stored in a semantic graph database (Neo4j[1]), allowing transitive similarity queries and historical reasoning.

Evaluation is carried out using annotated Jira[2] issue ticket data from two commercially successful game titles developed by Ubisoft[3], selected for their large user base, frequent updates, and extensive Jira logs made available for academic research, together with real Reddit feedback collected over two years.

Research motivation and novelties. Prior work in app review mining has made significant progress in sentiment analysis by Guzman and Maalej [1], topic extraction by Fu et al. [2], and bug detection by Maalej and Nabil [3]. However, these methods often operate in isolation and lack real-time integration into developer workflows. Systems such as AR-Miner by Chen et al. [4] and DIVER by Gao et al. [5] analyze issues post-hoc or offline. DeepMatcher by Häring et al. [6] proposed neural linking between review text and bug reports, but did not automate issue creation. CASPAR by Dina et al. [7] introduced the concept of extracting action-response patterns, yet lacked integration into real-world toolchains.

Our system builds on these ideas and operationalizes them in a unified, production-ready pipeline that integrates recent advances in both NLP and DevOps. It combines fast and slow inference stages for scalable processing, employs LLaMA 3.3 70B for summarization and reasoning, and utilizes semantic similarity graphs (via Neo4j) to track feedback clusters and temporal trends. Through component classification and priority scoring, the system enables structured feedback routing and supports real-time classification within development workflows.

Research questions (RQs). To guide our investigation and align our evaluation with the system's goals, we define the following research questions:

- **RQ1:** How accurately can the proposed system match user feedback to existing Jira issues using semantic and component-level similarity?
- **RQ2:** What is the throughput and resource cost of the pipeline at production-level Reddit ingestion rates?
- **RQ3:** How do developers perceive the usability of auto-generated tickets derived from user feedback?

Contributions. The contributions of this paper are the following: (a) We present a fully automated DevOps-integrated pipeline that continuously ingests user feedback and links feedback to existing issues or creates new ones based on semantics and component-level similarity with minimal human intervention. (b) In contrast to previous work, our approach unifies feedback ingestion, sentiment-driven prioritization, semantic deduplication, and issue generation into a real-time, scalable, and cost-effective solution for feedback-driven software development. (c) The proposed system utilizes sentiment analysis and natural language summarization to transform unstructured user comments into reproducible user stories, improving clarity and developer actionability. (d) We implement a semantic similarity mechanism that cross-references new feedback with existing issue tracker entries and identifies duplicates and unresolved bugs. (e) We release an open-source prototype of the system at `https://github.com/unibuc-cs/IssuesDetectionWithLLMs`, designed with a modular architecture to support extensibility, reproducibility, and further research. The evaluation was carried out using Ubisoft industry data and discussions with other partners such as Gameloft[4].

2. Related Work

A substantial body of work explores the use of natural language processing to mine user feedback for software maintenance and quality improvement. In their work, Chen et al. [4] introduced AR-Miner, a system that extracts relevant and informative reviews from noisy app store data using filtering, topic modeling, and ranking techniques. Their approach highlights actionable user complaints and includes visual analytics to aid developer navigation.

[1] `https://neo4j.com`
[2] `https://jira.atlassian.com`
[3] `https://ubisoft.com`
[4] `https://gameloft.com`

Our method is also related to the area of crowd-sourced requirements engineering, which investigates how user-generated feedback from public platforms can support system evolution. While many contributions in this field focus on classification or clustering, they often lack support for full automation from raw feedback to issue tracking. For example, Guzman and Maalej [1] proposed a fine-grained sentiment analysis framework that associates emotional polarity with specific app features, enhancing developer prioritization of feature updates.

Similarly, Maalej and Nabil [3] designed a supervised classification system to distinguish between bug reports, feature requests, and praise in app reviews. Their model supports triage workflows by assigning each review to the appropriate engineering tasks.

Gao et al. [5] presented DIVER, an industry-deployed pipeline that continuously monitors app reviews to surface emerging issues using unsupervised clustering and temporal trend analysis.

Li et al. [8] proposed RepRev, a system capable of automatically reproducing app crashes by parsing user reviews and simulating GUI behavior. Their work bridges the gap between unstructured feedback and concrete failure reproduction. Along similar lines, Dina et al. [7] developed CASPAR, a framework that extracts user problem narratives as action-reaction sequences, enabling clearer bug reproduction and categorization.

Häring et al. [6] tackled semantic review linkage by developing DeepMatcher, a neural model that aligns user reviews with bug reports in issue trackers. This facilitates deduplication and traceability across development pipelines.

In the domain of feature-centric review analysis, Fu et al. [2] introduced KEFE, a framework that extracts high-value feature mentions from reviews. Their method integrates app metadata with feedback to support product planning. Tang et al. [9] expanded this idea by proposing BugRMSys, a system that leverages semantic similarity to match bugs across related apps, fostering cross-project learning.

A complementary direction is explored by Lima et al. [10], who analyzed the temporal evolution of reported issues across releases. Their work emphasizes the importance of rapid feedback response to maintain user satisfaction and application reputation.

Although these contributions address important subproblems—such as sentiment classification, bug reproduction, and issue linking—they generally tackle them in isolation. Our system advances the state of the art by offering an integrated pipeline that spans user feedback ingestion, classification, summarization, and automated Jira issue creation. The resulting architecture supports real-time operations and is readily deployable in production DevOps environments.

3. Methods

The proposed solution introduces a multi-stage pipeline for automating issue tracking from user feedback, integrating natural language processing, semantic similarity reasoning, and DevOps orchestration. Feedback is collected from Reddit, filtered, summarized, semantically embedded, classified into functional components, and matched against existing Jira issues. Depending on similarity and component alignment, feedback is either appended to an existing ticket or used to generate a new one. A semantic graph of feedback-to-issue relations is maintained to support transitive similarity queries, feedback clustering, and historical traceability.

The pipeline leverages the LLaMA 3.3 70B chat model [11], referred to as LLM, for both summarization and reasoning tasks. This model was selected for its open-source accessibility, state-of-the-art reasoning performance, and compatibility with low-latency inference engines such as vLLM. We also tested smaller or similarly sized models, but they underperformed on complex summarization or structured output tasks. For semantic embedding and similarity computations, the `instructor-large` model from the Instructor family [12] is used. This model is instruction-tuned, enabling it to generate task-specific embeddings that better reflect the intent and structure of feedback content. It is fully compatible with similarity-based operations such as cosine distance and nearest-neighbor queries. The choice of these models balances quality and efficiency, offering competitive performance in real-time systems.

Overall, the architecture is designed to be modular, with components and models easily interchangeable. It requires no labeled training data from the client (e.g., development team adopting the system); all raw feedback is ingested and analyzed through learned or rule-based mappings to internal taxonomies. Clients are only responsible for providing domain configuration files, such as component definitions, tag aliases, and Jira field mappings, while all semantic processing, similarity computation, and issue triaging are handled automatically.

The core modules of this system are described in the following subsections and are reflected in the procedural logic shown in Figure 1 and Listing 1. Concrete examples of Reddit feedback and their full end-to-end processing,

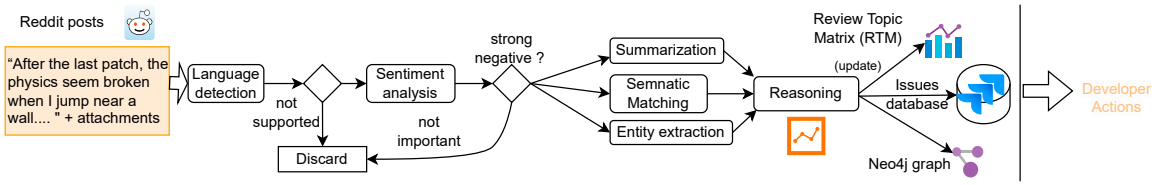

Fig. 1: End-to-end LLM-augmented pipeline for automated issue tracking.

covering all pipeline stages, including discarding, semantic linking, and ticket creation, are provided in the appendix at https://tinyurl.com/2rdm7fw7.

3.1. Feedback Instance Representation

Before ingestion, each client of the pipeline must define a domain-specific set of issue categories aligned with its internal Jira structure. Let $\mathcal{T}$ denote the client-defined taxonomy of tags, such that each tag $\tau \in \mathcal{T}$ corresponds to a component or feature tracked in the client's issue management system. These tags are used later for classification and topic aggregation. The components (also detailed in Section 3.2 and exemplified in Table 2) represent logical or architectural subsystems responsible for particular functionalities within the application. These can be independently monitored, classified, or debugged. After feedback is embedded, it is classified into one or more components using $\mathcal{T}$. This allows routing feedback to the correct team: e.g., UI bugs go to the UI/UX team.

Let $\mathcal{D}$ denote the database containing all feedback entries. Each record $F_i \in \mathcal{D}$ represents an atomic unit of feedback derived from Reddit, transformed through preprocessing operations. Formally, an instance is defined as:

$$\begin{aligned} &F_i = \{\mathrm{ID}_i, \mathrm{Time}_i, \mathrm{Source}_i, \mathrm{Text}_i, \rho_i, \mathrm{Meta}_i\} \text{ and} \\ &\mathrm{Meta}_i = \{\mathrm{Lang}_i, \mathrm{Summary}_i, \mathrm{Entities}_i, \mathrm{Tags}_i, \mathrm{Attachments}_i, \mathrm{Status}_i, \mathrm{SimilarityEdges}_i, \mathrm{Components}_i\ \mathrm{Origin}_i\} \end{aligned} \tag{1}$$

where: (a) ID_i: unique identifier for the Reddit feedback entry, (b) Time_i: timestamp at which the comment or post was created, (c) Source_i: metadata about the Reddit source (e.g., subreddit, thread ID) (d) Text_i: full raw content of the feedback. (e) ρ_i: sentiment score in the continuous range $[-5, 5]$. Scores below zero imply negative sentiment, scores above zero imply positive sentiment, and values near zero correspond to neutral feedback. Moreover, Meta_i contains language, summary, extracted entities, assigned tags, attached media, current status, semantic similarity links to issues, and predicted component distribution. The field $\mathrm{SimilarityEdges}_i$ (1) encodes top-k links to similar issues or feedback nodes based on the semantic graph $\mathcal{G}$, used for deduplication and trend analysis. The $\mathrm{Components}_i$ field stores the probability distribution over the component taxonomy $\mathcal{T}$, obtained from classifier $\mathcal{F}$ as described in Section 3.2. The last field Origin_i supports traceability and interpretability by storing diagnostic information on how the feedback instance was processed and mapped by the system. This may include the ID of the LLM used, thresholds applied during filtering, the timestamp of classification, or the pipeline version. This metadata is especially valuable for debugging or for human-in-the-loop workflows where developers may need to audit or validate system decisions.

Technically, the user feedback is collected using the Python Reddit API Wrapper (PRAW) [13], which provides stable access to Reddit endpoints, including posts, comment threads, metadata, and multimedia attachments. Threads are fetched manually or periodically (e.g., hourly-to-weekly in our experiments, Section 4) via scheduling triggers (using CRON and Kubernetes Jobs [14]). Posts are filtered using the taxonomy of predefined tags, reflecting known issue categories within the Jira tracker. Data ingestion proceeds through Apache Kafka, a high-throughput distributed event streaming platform. Each Reddit comment is serialized into Kafka topics that serve downstream inference modules. In our context, Kafka ensures persistence, order-preservation, and scalable replayability of feedback messages. In our architecture, producers emit structured JSON payloads to the Kafka bus, while consumers represent Kubernetes-based NLP microservices. These consumers can be scaled horizontally based on partition count and traffic.

3.2. Internal Knowledge Representation

The internal representation defines structures for storing domain concepts, enriched feedback metadata, and outputs of intermediate classifiers. Central to this representation is the feedback corpus $\mathcal{D}$, where each instance F_i is structured as defined in Equation (1).

Components. This defines the client-defined taxonomy $\mathcal{T}$ of known subsystems or architectural elements relevant to the application (e.g., Table 2). Each component $c_j \in \mathcal{T}$ is defined as a tuple:

$$Component_j = \{\text{Name}_j, \text{Alias}_j, \text{DocURL}_j, \text{Subsystem}_j\}, \tag{2}$$

where Name_j is the canonical label (e.g., "UI Flow"), Alias_j contains known lexical variants (e.g., "menus", "HUD"), DocURL_j provides a pointer to internal documentation, and Subsystem_j encodes its placement in the software architecture (e.g., "Graphics Engine::Input Layer").

Component Probability Distribution. Component classification is performed via a probabilistic mapping function `ClassifyComponent` that maps a feedback embedding to a vector of probabilities over the components:

$$\texttt{ClassifyComponent}(e_{F_i}) = [p(c_1 \mid e_{F_i}), ..., p(c_m \mid e_{F_i})] \tag{3}$$

where each $p(c_j \mid e_{F_i}) \in [0, 1]$ and the vector is normalized to sum to 1. The resulting component distribution is stored as $\text{Components}_i \in \text{Meta}_i$.

Review-Topic Matrix (RTM). To support trend analysis and heatmapping of architectural stress, we define a Review-Topic Matrix (RTM) that aggregates component-level probabilities across time:

$$\text{RTM}_t(c_j) = \sum_{F_i \in \mathcal{D}_t} p(c_j \mid e_{F_i}) \tag{4}$$

where $\mathcal{D}_t \subset \mathcal{D}$ is the set of feedbacks in time window t, and $p(c_j \mid e_{F_i})$ is the component assignment score. The RTM can be queried to identify which subsystems are under high reporting load. A concrete example is shown in Figure 2, where spikes over "Graphics" may indicate a recent rendering bug.

Semantic Similarity Graph. Semantic relations between feedback entries F_i and Jira issues J_k are stored in a dynamic graph structure $\mathcal{G} = (V, E)$ using a graph database backend (such as Neo4j). Vertices V correspond to feedback instances and Jira issues, while edges E encode semantic similarity scores σ_{ik} between F_i and J_k.

Upon processing each feedback entry F_i, semantic similarity scores σ_{ik} are computed with respect to existing issues J_k using cosine similarity over contextualized embeddings. Threshold parameters γ_s and γ_e are defined to control matching behavior, where γ_s denotes the lower similarity threshold for possible contextual relevance, and γ_e represents the confidence matching for deduplication. To handle partially similar feedback (i.e., $\gamma_s < \sigma_{ik} \leq \gamma_e$), we incorporate a component alignment heuristic. Let $\vec{p}_{F_i}$ denote the component probability distribution of F_i (from `ClassifyComponent`), and $\vec{q}_{J_k}$ be the aggregated (e.g., median or mean) component vector from previously linked feedback associated with issue J_k. Their alignment is computed as shown in Eq. 5.

$$\text{ComponentSim}(F_i, J_k) = \cos(\vec{p}_{F_i}, \vec{q}_{J_k}) \tag{5}$$

Although this uses the same similarity function as σ_{ik}, it operates in the component probability space rather than the semantic embedding space. If $\text{ComponentSim}(F_i, J_k) > \delta$ for threshold δ, the feedback is appended as a supporting comment to J_k; otherwise, a new issue is instantiated.

Semantic match are recorded as edges: `(:Feedback)-[:SIMILAR_TO {similarity: `σ_{ik}`}]->(:JiraIssue)`. This representation allows transitive reasoning, top-k similarity queries, and dynamic deduplication across feedback entries. Graph features can be periodically queried in practice to: (a) Identify densely connected clusters of feedback, suggesting emerging systemic issues or hot topics. (b) Flag isolated yet high-priority feedback for new issue creation. When $\sigma_{ik} > \gamma_e$, a corresponding edge is created in Neo4j, and F_i is linked to J_k. (c) Detect feedback that is moderately similar (i.e., $\gamma_s < \sigma_{ik} \leq \gamma_e$) to multiple issues, marking it as an unresolved discussion.

This mechanism is tightly coupled with the inference loop outlined in Listing 1. Parameter values γ_s, γ_e, and δ are introduced and tuned empirically, and documented in Section 4.

3.3. Action Flow Extraction from Feedback

This feature enables granular analysis of feedback by reconstructing user behavior paths and improving issue reproducibility and component-level diagnostics. To convert unstructured user feedback into structured sequences of actions that may have caused a failure, a hybrid pipeline that combines classical NLP techniques with modern LLMs is adopted. The steps are as follows:

A. Preprocessing and sentence segmentation. Each feedback instance F_i is first segmented into a set of candidate sentences $S_i = \{s_1, s_2, \ldots, s_n\}$ using rule-based sentence tokenization. Sentences are heuristically filtered to retain those expressing user-performed actions or observed software responses.

B. POS tagging and dependency parsing. Each sentence s_k is parsed using spaCy [15], a syntactic parser, to extract verb-object relations and temporal indicators. This yields action tuples of the form:

$$A_k = (\text{subject}, \text{verb}, \text{object}, \text{temporal info}) \tag{6}$$

C. Lexical anchoring with component keywords. Verbs and objects from A_k are matched against component-specific keyword lists from Alias_j field in the component's definition, Eq. 2. This lexical anchoring maps actions to functional areas and reduces semantic drift.

D. Temporal ordering. Temporal cues (e.g., "then", "after", "before") and narrative structure are used to reconstruct the ordered chain:

$$A_1 \rightarrow A_2 \rightarrow \cdots \rightarrow R_k \tag{7}$$

where R_k denotes the resulting failure or unexpected behavior (e.g., "User presses inventory → selects the shovel → UI freezes").

The action sequence is obtained using the LLM with the features obtained in the previous steps and using the methods suggested in [16]. The finalized sequence is stored in F_i's, Meta `actions` field, and associated with relevant components via semantic and keyword-based matching.

3.4. Text Processing and Sentiment Analysis

Once a Reddit feedback entry F_i is ingested, it is passed through a multi-stage NLP pipeline designed to extract actionable semantics. Each stage incrementally enriches the metadata Meta_i associated with F_i, cf. Equation 1.

Language identification is first performed using FastText [17], populating Lang_i. This early step ensures that unsupported languages are excluded or rerouted to fallback pipelines, improving processing efficiency.

Sentiment analysis is conducted in two stages to balance latency with accuracy. The first-stage classifier is a fine-tuned RoBERTa model [18], optimized for binary or ternary sentiment classification over short-form, user-generated content. If the output indicates negative or ambiguous sentiment, the entry is passed to a second-stage LLM, which uses a structured prompt to regress the feedback into a calibrated scalar sentiment score $\rho_i \in [-5, 5]$. This value is stored directly in the feedback instance. Scores below -2 are treated as highly negative, while scores above 2 indicate strongly positive sentiment. Intermediate values are interpreted as neutral or weak polarity. The scalar score ρ_i is later used in prioritization logic (Eq. 9) to influence issue triage and ticket ranking.

For feedback classified as strongly negative, a summarization step is applied using the same LLM in summarization mode. This produces Summary_i, which is used for Jira issue titles and downstream component classification. Named entity recognition (NER) and keyword extraction are then applied. Entities_i are extracted via spaCy [15], optionally extended with domain-specific vocabularies (e.g., game-specific terminology in our evaluation). Keywords are identified using TF-IDF [19] and saliency scores derived from attention mechanisms in the embedding model. These features are mapped to the client-defined taxonomy $\mathcal{T}$ to generate structured tags Tags_i.

Overall, this pipeline transforms raw text into semantically enriched representations, enabling accurate routing, prioritization, and ticket generation in downstream modules.

3.5. Ticket Generation and Prioritization

The next step is to determine whether feedback F_i should result in a new Jira issue or if it aligns with an existing one. When no suitable match is found, a new issue is created. The issue encapsulates structured elements:

```
for F_i in fetch_reddit_posts(t_start, t_end):
  if detect_language(F_i.text) not in supported_languages:
    continue
  rho_i = classify_sentiment(F_i.text)
  if rho_i > sentiment_threshold:
    continue
  F_i.Meta.Summary = summarize_text(F_i.text)
  F_i.Meta.Entities = extract_entities(F_i.text)
  F_i.Meta.Tags = match_tags(F_i.Meta.Entities)
  embedding = compute_embedding(F_i.text)
  F_i.Meta.Components = classify_components(embedding)
  update_review_topic_matrix(F_i.timestamp, F_i.Meta.Components)

  similar_issues = query_similar_issues(embedding, threshold=γ_s)

  if not similar_issues:
    new_issue = create_jira_ticket(F_i.Meta.Summary, F_i.text, F_i.Meta.Tags, ρ_i, F_i.Meta.Attachments)
    store_in_graph(F_i.id, new_issue.id, similarity=1.0)
    set_feedback_status(F_i.id, "linked_new")
  else:
    best_match = similar_issues[0]
    sigma = best_match.similarity

    if sigma > gamma_e:
    set_feedback_status(F_i.id, "duplicate_discarded")
  else:
    q_components = get_component_distribution(best_match.id)
    overlap = cosine_similarity(F_i.Meta.Components, q_components)

    if overlap > delta:
      append_comment_to_jira(best_match.id, F_i.text)
      store_in_graph(F_i.id, best_match.id, similarity=sigma)
      set_feedback_status(F_i.id, "linked_existing_component_match")
    else:
      new_issue = create_jira_ticket(F_i.Meta.Summary, F_i.text, F_i.Meta.Tags, rho_i, F_i.Meta.Attachments)
      store_in_graph(F_i.id, new_issue.id, similarity=1.0)
      set_feedback_status(F_i.id, "linked_new_different_component")
```

Listing 1: Pseudocode for Reddit feedback processing, ticket assignment with semantic and component matching.

$$\text{Ticket}_i = \{\text{Title: Summary}_i,\ \text{Description: Text}_i,\ \text{Tags: Tags}_i,\ \text{Attachments}_i\} \tag{8}$$

Prioritization score P_i is computed from three main signals: sentiment (S_i), report frequency (R_i), and user credibility (U_i):

$$P_i = \alpha S_i + \beta R_i + \gamma U_i \tag{9}$$

Here, S_i is a rescaled version of the sentiment score ρ_i. R_i reflects how frequently a similar issue was reported in a recent time window. U_i quantifies user trustworthiness using Reddit metadata (e.g., karma, account age). Weights α, β, and γ are optimized via cross-validation on historical Jira labels using grid search and Bayesian optimization. The resulting scores P_i guide issue ordering and triaging within the DevOps workflow.

The pseudocode following the presented steps above is shown in Listing 1.

3.6. Human-in-the-Loop Corrections and Continuous Learning

The system supports continuous learning by capturing implicit feedback from developers interacting with the AI-generated tickets. When developers reject, edit, merge, or reassign feedback-based tickets, these actions are logged as high-quality supervisory signals. Unlike passive labeling, these decisions represent contextualized, domain-expert judgments aligned with internal project goals. For instance, if the AI assigns a post to the "Graphics" component but a developer reclassifies it under "UI/UX" during triage, the correction can be fed back into the component classifier as training data. Similarly, if a suggested semantic match to an existing issue is rejected and a new Jira ticket is created instead, this provides a hard negative signal for future matching. This human-in-the-loop integration can provide

correction signals used to fine-tune: a) Sentiment classifiers (was the urgency misjudged?), b) Component classifiers (was the bug misattributed?), c) Semantic matchers (did we propose a wrong existing ticket?)

4. Evaluation

This section evaluates the system's effectiveness, scalability, and practical utility across three key research questions. Experiments are conducted using the dataset introduced by Paduraru et al. [20], which includes developer-maintained Jira logs from Ubisoft titles such as *Assassin's Creed*[5] and *The Crew*[6], spanning 2019–2021. The dataset includes approximately 8,500 Jira tickets covering gameplay issues, performance regressions, and design feedback.

In parallel, a collection of ~12,000 Reddit posts was curated from relevant subreddits (e.g., `r/assassinscreed`, `r/thecrew2`) during the same period. Language filtering and deduplication yielded 10,430 unique feedback entries. Among them, 3,860 entries (~37%) were classified as strongly negative ($\rho_i < -2$), and thus subject to downstream summarization and issue generation workflows, as described in Section 3. The evaluation primarily focuses on the matching and triaging components of the pipeline, as no complete ground truth for full ticket creation exists. Nevertheless, ticket generation logic is implemented and described in Section 3.5 and Listing 1. Additionally, RQ3 evaluates the system's ability to generate new tickets by assessing the quality of summaries produced for unmatched user feedback, as reviewed by eight software engineers.

Using the RTM data structure, Figure 2 illustrates a hotspot analysis conducted over five weeks following the release of a major game update. Notably, a spike in feedback was observed in the *Graphics* and *Multiplayer* components, suggesting emerging issues related to rendering and online gameplay stability. Table 2 provides concrete examples of component mappings based on the client-defined taxonomy $\mathcal{T}$, highlighting how representative feedback is classified and linked to internal system components.

All pipeline parameters, including thresholds for semantic similarity and component overlap, are summarized in Table 1. These values were selected either empirically or tuned via cross-validation on historical Jira issue data, with the goal of maximizing precision and recall in semantic filtering. Hyperparameters α, β, and γ were selected through grid search over validation data using historical ticket urgency labels as ground truth. Guided by this configuration and the methodology outlined in Section 3, the evaluation aims to answer the three research questions described in Section 1, each designed to validate key components of the proposed pipeline.

Although several prior works have addressed individual components of the feedback-to-ticket workflow, such as sentiment classification [1], review clustering [4], or issue deduplication [6], to our knowledge, there exists no publicly available baseline that implements and evaluates a comparable end-to-end DevOps-integrated system. As such, our evaluation strategy focuses on decomposing the pipeline into measurable subsystems and addressing each through a scoped research questions. This modular evaluation design enables empirical validation of the pipeline's core capabilities in the absence of full-system baselines.

RQ1: How accurately can the proposed system match user feedback to existing Jira issues using semantic and component-level similarity? (Semantic and Component-Level Matching Accuracy analysis)
Matching accuracy is assessed by comparing system predictions to manually annotated Reddit–Jira pairs. A match is considered correct if it falls in the top-k nearest semantic neighbors (with $k = 5$) and shares a component label from the taxonomy $\mathcal{T}$. On a stratified sample of 500 Reddit entries, the system achieves top-1 matching accuracy of 82.4%, and top-3 accuracy of 91.1%.

Incorporating component alignment via the ComponentSim function led to a 27% reduction in false-positive links in the mid-similarity range ($\gamma_s < \sigma_{ik} \leq \gamma_e$), compared to a baseline using only `instructor-large` embeddings. This shows that semantic embeddings alone can confuse issues from different subsystems that share vocabulary, and that joint alignment improves traceability (correct linkage to subsystem) and deduplication (avoiding redundant issues).

RQ2: What is the throughput and resource cost of the pipeline under production-level Reddit ingestion rates? (Pipeline Throughput and Resource Cost analysis) The pipeline was benchmarked on 12,000 Reddit posts per day.

[5] https://www.ubisoft.com/en-gb/game/assassins-creed
[6] https://store.steampowered.com/app/646910/The_Crew_2/

Parameter	Value / Description
ρ_i	Sentiment score $[-5, 5]$
γ_s	0.75 – Lower threshold for semantic similarity (soft match)
γ_e	0.90 – Upper threshold for semantic similarity (confident match)
δ	0.80 – Component similarity threshold (ComponentSim)
α	0.5 – Sentiment weight in P_i
β	0.3 – Repetition weight in P_i
γ	0.2 – User credibility weight in P_i
t	1 day (release), 7 days (stable)
k	5 – Top-k semantic neighbors

Table 1: Hyperparameters and thresholds used throughout the feedback processing pipeline.

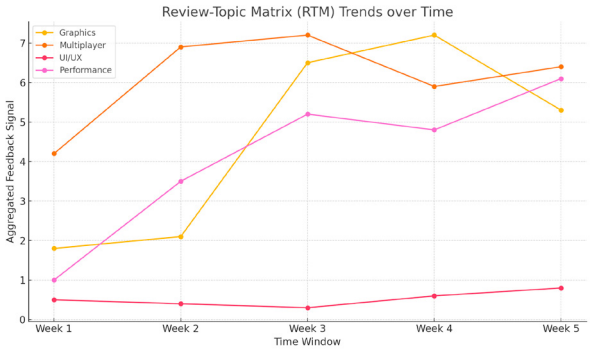

Fig. 2: Review-Topic Matrix (RTM) evolution across key components over five weekly intervals.

User Review Text	Component	Reasoning	Aliases (Keywords)
Every time I join a multiplayer match, it lags like crazy and disconnects.	Multiplayer	"multiplayer", "lag", "disconnects"	multiplayer, network, online, matchmaking
The shadows and reflections look broken after the last update.	Graphics	Visual rendering defects	shadows, lighting, reflections, textures
Menus don't respond when I press back, and the inventory disappears.	UI/UX	GUI interaction issues	UI, HUD, menu, screen
The boss AI keeps walking into walls and doesn't notice the player	AI	NPC pathfinding bugs	AI, bot, navigation, enemy logic
I'm running it on a high-end rig, but I still get FPS drops in big fights.	Performance	Framerate drops	FPS, optimization, lag, memory
After 3 hours of gameplay, it overheated my phone and crashed.	Stability	Thermal and crash failure	crash, overheating, freeze
I like the game, but it's way too easy after the first few levels.	Gameplay Balance	Difficulty issues	difficulty, balance, challenge
There's no music during boss fights, and the sound cuts out randomly.	Audio	Missing sound effects	audio, sound, mute, volume

Table 2: Component mapping examples from user feedback in game development. Components are derived from the taxonomy $\mathcal{T}$ and guide feedback routing and RTM updates.

The first-stage components (TF-IDF, FastText, RoBERTa) were deployed on a 20 vCPU Intel Xeon node, achieving latency under 10ms per post. Due to strong negative sentiment or low classification confidence, approximately 37% (~4,400) of posts were escalated to the LLM-based tier. These were processed entirely on 8×H100 GPUs, where LLaMA 3.3 70B Chat (via vLLM) performed summarization with a batch size of 4, and `instructor-large` generated embeddings at batch size 32 (under 200ms/post). The system sustained throughput of ~3,000 enriched posts per hour. Total GPU usage averaged ~13.8 GPU-hours/day, corresponding to an estimated cost of $39–$45/day at standard H100 on-demand pricing, confirming the pipeline's scalability and feasibility for production-level feedback triage.

RQ3: How do developers perceive the usability of auto-generated tickets derived from user feedback?
To evaluate the quality of new-ticket generation and summarization, eight Ubisoft developers reviewed 30 auto-generated Jira tickets—half containing Summary_i and half without. Tickets with summaries were rated as clearer in 92% of cases, and triage time was reduced by an average of 34.6%. Participants noted greater clarity in issue titles and a lower cognitive burden when assessing urgency or relevance. These results support the effectiveness of LLM-based summarization in enhancing developer productivity, particularly in high-volume feedback scenarios.

Limitations. While the system supports full end-to-end Jira ticket generation, the evaluation focuses primarily on semantic and component-level alignment due to the absence of a labeled ground truth dataset for new issue creation. As a result, the effectiveness of ticket generation was assessed qualitatively through developer feedback rather than quantitative metrics. Furthermore, Reddit feedback is often informal, ambiguous, or redundant over time, which may impact the clarity and reproducibility of generated issues. The current implementation also assumes monolingual (English) input; supporting multilingual feedback remains a target for future work.

5. Conclusion and future work

This paper presents a modular and scalable pipeline for automating issue tracking from user-generated feedback, combining natural language processing, semantic matching, and DevOps integration. By leveraging LLMs for summarization, difficult sentiment analysis cases, and component-aware similarity reasoning, the system accurately links Reddit posts to existing Jira issues or generates new tickets, enhancing traceability and reducing manual triage overhead. Empirical evaluation on real-world game development data confirms the pipeline's effectiveness, scalability, and practical utility in production settings.

While the current system focuses primarily on identifying and routing functional issues (such as gameplay bugs, interface glitches, and AI failures), future extensions may address the often-overlooked category of non-functional feedback (e.g., reports concerning usability challenges, battery consumption, and other quality-of-experience factors).

Acknowledgements This research was supported by the projects "Romanian Hub for Artificial Intelligence - HRIA", Smart Growth, Digitization and Financial Instruments Program, 2021-2027, MySMIS no. 334906 and a grant of the Ministry of Research, Innovation and Digitization, CNCS/CCCDI-UEFISCDI, project no. PN-IV-P8-8.1-PRE-HE-ORG-2023-0081, within PNCDI IV.

References

[1] E. Guzman, W. Maalej, App stores: Review analysis and mining, in: Proc. of the Requirements Engineering Conference (RE'14), IEEE, 2014, pp. 43–52.
[2] Z. Fu, Z. Li, H. Zhang, S. Wang, Y. Zhou, KEFE: Key feature extraction from user reviews for mobile apps, in: Proc. of the 43rd International Conference on Software Engineering (ICSE'21), ACM, 2021, pp. 881–893.
[3] W. Maalej, H. Nabil, Automatic classification of app reviews into bug reports and feature requests, in: Proc. of the 23rd International Requirements Engineering Conference (RE'15), IEEE, 2015, pp. 116–125.
[4] N. Chen, J. Lin, S. Hoi, X. Xiao, B. Zhang, M. Zhou, AR-Miner: Mining informative reviews for developers from mobile app marketplace, in: Proc. of the 36th International Conference on Software Engineering (ICSE'14), ACM, 2014, pp. 767–778.
[5] C. Gao, J. Zhai, X. Gu, D. Xu, L. Zhang, T. Wang, Identifying emergent user issues from app reviews, in: Proc. of 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP'19), IEEE, 2019, pp. 11–20.
[6] M. Häring, A. Al-Subaihin, M. Aniche, A. Zaidman, DeepMatcher: Linking user reviews to source code artifacts using deep learning, in: Proc. of the 43rd International Conference on Software Engineering (ICSE'21), 2021, pp. 431–442.
[7] M. Dina, W. Maalej, A. Nolte, CASPAR: Extracting user problem stories from app reviews, in: Proc. of the 42nd International Conference on Software Engineering (ICSE'20), ACM, 2020, pp. 758–770.
[8] Y. Li, Y. Li, X. Zhang, G. Meng, Z. Chen, J. Li, RepRev: Automatically reproducing crash reports from user reviews, in: Proc. of the 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP'20), IEEE, 2020, pp. 12–21.
[9] Y. Tang, C. Gao, Y. Li, T. Wang, X. Zhang, BugRMSys: Discovering similar bugs across apps from user reviews using semantic similarity, in: Proc. of the 45th International Conference on Software Engineering (ICSE'23), ACM, 2023, pp. 899–911.
[10] B. Lima, P. Oliveira, M. T. Valente, Temporal dynamics of user-reported issues in mobile apps, in: Proc. of the International Conference on Software Maintenance and Evolution (ICSME'24), IEEE, 2024.
[11] A. Grattafiori, et al., The Llama 3 herd of models (2024). arXiv:2407.21783.
[12] H. Su, W. Shi, J. Kasai, Y. Wang, Y. Hu, M. Ostendorf, W. tau Yih, N. A. Smith, L. Zettlemoyer, T. Yu, One embedder, any task: Instruction-finetuned text embeddings (2023). arXiv:2212.09741.
[13] S. Mer, S. Saxena, A. S. Pundir, Revolutionizing sentiment analysis: Accelerated data science approaches for reddit submissions, in: Revolutionizing of the 2024 IEEE INDISCON conference, 2024, pp. 1–6.
[14] I. Harichane, S. A. Makhlouf, G. Belalem, KubeSC-RTP: Smart scheduler for kubernetes platform on CPU-GPU heterogeneous systems, Concurrency and Computation: Practice and Experience 34 (21) (2022) e7108.
[15] M. Honnibal, I. Montani, spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing, To appear (2017).
URL https://spacy.io
[16] S. Wang, et al., Can LLMs reason with rules? logic scaffolding for stress-testing and improving LLMs, in: Proc. of Association for Computational Linguistics (ACL'24), 2024, pp. 7523–7543.
[17] A. Joulin, E. Grave, P. Bojanowski, T. Mikolov, Bag of tricks for efficient text classification, in: Proc. of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL'17), 2017, pp. 427–431.
[18] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, RoBERTa: A robustly optimized BERT pretraining approach, arXiv preprint arXiv:1907.11692 (2019).
[19] J. Ramos, Using TF-IDF to determine word relevance in document queries, in: Proc. of the First Instructional Conference on Machine Learning (ICML), Vol. 242, 2003, pp. 29–48.
[20] C. Paduraru, M. Paduraru, C. C. Patilea, Task distribution and human resource management using reinforcement learning, in: 2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW), 2021, pp. 96–101.